

3567. Minimum Absolute Difference in Sliding Submatrix

Solved 

Medium

 Topics

 Companies

 Hint

You are given an $m \times n$ integer matrix `grid` and an integer `k`.

For every contiguous $k \times k$ **submatrix** of `grid`, compute the **minimum absolute** difference between any two **distinct** values within that **submatrix**.

Return a 2D array `ans` of size $(m - k + 1) \times (n - k + 1)$, where `ans[i][j]` is the minimum absolute difference in the submatrix whose top-left corner is (i, j) in `grid`.

Note: If all elements in the submatrix have the same value, the answer will be 0.

A submatrix $(x1, y1, x2, y2)$ is a matrix that is formed by choosing all cells `matrix[x][y]` where $x1 \leq x \leq x2$ and $y1 \leq y \leq y2$.

Example 1:

Input: `grid = [[1,8],[3,-2]]`, `k = 2`

Output: `[[2]]`

Explanation:

- There is only one possible `k x k` submatrix: `[[1, 8], [3, -2]]`.
- Distinct values in the submatrix are `[1, 8, 3, -2]`.
- The minimum absolute difference in the submatrix is `|1 - 3| = 2`. Thus, the answer is `[[2]]`.

Example 2:

Input: `grid = [[3,-1]]`, `k = 1`

Output: `[[0,0]]`

Explanation:

- Both `k x k` submatrix has only one distinct element.
- Thus, the answer is `[[0, 0]]`.

Example 3:

Input: `grid = [[1,-2,3],[2,3,5]], k = 2`

Output: `[[1,2]]`

Explanation:

- There are two possible $k \times k$ submatrix:
 - Starting at $(0, 0)$: `[[1, -2], [2, 3]]`.
 - Distinct values in the submatrix are `[1, -2, 2, 3]`.
 - The minimum absolute difference in the submatrix is $|1 - 2| = 1$.
 - Starting at $(0, 1)$: `[[-2, 3], [3, 5]]`.
 - Distinct values in the submatrix are `[-2, 3, 5]`.
 - The minimum absolute difference in the submatrix is $|3 - 5| = 2$.
- Thus, the answer is `[[1, 2]]`.

Constraints:

- `1 <= m == grid.length <= 30`
- `1 <= n == grid[i].length <= 30`
- `-105 <= grid[i][j] <= 105`
- `1 <= k <= min(m, n)`

Python:

class Solution:

```
def minAbsDiff(self, grid: List[List[int]], k: int) -> List[List[int]]:  
    m, n = len(grid), len(grid[0])  
    res = [[0] * (n - k + 1) for _ in range(m - k + 1)]
```

```

for i in range(m - k + 1):
    for j in range(n - k + 1):
        kgrid = []
        for x in range(i, i + k):
            for y in range(j, j + k):
                kgrid.append(grid[x][y])
        kmin = float("inf")
        kgrid.sort()
        for t in range(1, len(kgrid)):
            if kgrid[t] == kgrid[t - 1]:
                continue
            kmin = min(kmin, kgrid[t] - kgrid[t - 1])
        if kmin != float("inf"):
            res[i][j] = kmin
return res

```

JavaScript:

```

var minAbsDiff = function (grid, k) {
    const m = grid.length,
          n = grid[0].length;
    const res = Array.from({ length: m - k + 1 }, () =>
        Array(n - k + 1).fill(0),
    );
    for (let i = 0; i + k <= m; i++) {
        for (let j = 0; j + k <= n; j++) {
            let kgrid = [];
            for (let x = i; x < i + k; x++) {
                for (let y = j; y < j + k; y++) {
                    kgrid.push(grid[x][y]);
                }
            }
            let kmin = Number.MAX_SAFE_INTEGER;
            kgrid.sort((a, b) => a - b);
            for (let t = 1; t < kgrid.length; t++) {
                if (kgrid[t] === kgrid[t - 1]) {
                    continue;
                }
                kmin = Math.min(kmin, kgrid[t] - kgrid[t - 1]);
            }
            if (kmin !== Number.MAX_SAFE_INTEGER) {
                res[i][j] = kmin;
            }
        }
    }
}

```

```
    return res;
};
```

Java:

```
class Solution {

    public int[][] minAbsDiff(int[][] grid, int k) {
        int m = grid.length;
        int n = grid[0].length;
        int[][] res = new int[m - k + 1][n - k + 1];
        for (int i = 0; i + k <= m; i++) {
            for (int j = 0; j + k <= n; j++) {
                List<Integer> kgrid = new ArrayList<>();
                for (int x = i; x < i + k; x++) {
                    for (int y = j; y < j + k; y++) {
                        kgrid.add(grid[x][y]);
                    }
                }
                int kmin = Integer.MAX_VALUE;
                Collections.sort(kgrid);
                for (int t = 1; t < kgrid.size(); t++) {
                    if (kgrid.get(t).equals(kgrid.get(t - 1))) {
                        continue;
                    }
                    kmin = Math.min(kmin, kgrid.get(t) - kgrid.get(t - 1));
                }
                if (kmin != Integer.MAX_VALUE) {
                    res[i][j] = kmin;
                }
            }
        }
        return res;
    }
}
```